

Apostila do Projeto Individual

SoundLab — Meu Player: Planejamento e Entregas de 6 Semanas

Período de execução: a partir da Aula 07 (Filas e Filas Circulares), conforme o calendário de aulas da disciplina.

Documento único que orienta a reta final de Estrutura de Dados I: da fila de execução à apresentação do player individual de cada aluno, construído sobre a base coletiva do SoundLab já implementada nas Aulas 01 a 06.

1. Visão Geral do Projeto Individual

Este documento organiza os Módulos 3 e 4 do plano de ensino de Estrutura de Dados I — árvores, árvore binária de busca, grafos, busca linear/binária, Big-O e ordenação — em torno de um único objetivo: cada aluno transforma o SoundLab, construído coletivamente da Aula 01 à Aula 06 (TAD Faixa, Acervo, Playlist com navegação dupla, Pilha de histórico), na sua própria versão completa do player, em seis semanas de oficina individual.

O formato segue a mesma lógica já usada em Estrutura de Dados II para o projeto final: até aqui, o SoundLab foi construído coletivamente, em ritmo de aula ao vivo, com toda a turma chegando ao mesmo código ao final de cada encontro. As seis semanas finais invertem esse eixo — a base de código (Faixa, Acervo, Playlist, Pilha) já é comum e está pronta; o que resta é decisão de projeto e ritmo individual de implementação, e por isso passa a ser avaliado aluno por aluno, em branches separadas do mesmo repositório.

Por que branches individuais, e por que agora

Da Aula 01 à Aula 06, todos os alunos trabalharam sobre o mesmo código, evoluindo-o junto em sala. A partir da Aula 07, cada aluno clona o repositório SoundLab e passa a trabalhar na sua própria branch, identificada por nome e matrícula — exatamente para que o código de um aluno nunca sobrescreva o de outro, já que a partir daqui cada player segue decisões próprias (como organizar o grafo de recomendação, qual critério de ordenação priorizar, etc.).

Observação: Nada do que já foi construído até a Aula 06 é descartado. Faixa, Acervo, Playlist e Pilha já existem no projeto e devem ser reaproveitados sem reescrita — a mesma regra de ouro seguida desde a Aula 01.

O que já existe (ponto de partida, Aula 06)

Camada	O que já está pronto
TAD Faixa	título, artista, duracao_seg, num_execucoes; métodos tocar() e duracao_formatada().
TAD Acervo	coleção de Faixas; adicionar, buscar(título) [busca linear], remover, len, iteração.
TAD Playlist	lista duplamente encadeada de Faixas; adicionar, inserir_em, remover_em, obter; navegação de reprodução comecar_reproducao / proxima / anterior, O(1) nos dois sentidos.
TAD Pilha	histórico de reprodução (LIFO); empilhar, desempilhar, topo, vazia — usada quando uma faixa é tocada direto do acervo, fora da playlist.

O player que as seis semanas constroem

Ao final da Semana 6, cada aluno deve ter um SoundLab individual capaz de:

1. Organizar fila de execução ("a seguir") e um modo de repetição em loop, sem depender da Playlist já existente — são estruturas novas, não extensões dela.
2. Indexar o acervo numa árvore binária e, a partir dela, numa árvore binária de busca ordenada por título, com busca e listagem alfabética por percurso em-ordem.
3. Recomendar faixas ("quem ouviu X também ouviu Y") a partir de um grafo de coocorrência construído sobre o histórico de reprodução.
4. Comparar, com números reais do próprio acervo, o custo da busca linear original contra uma busca binária (ou a ABB da Semana 2), relacionando o resultado à notação Big-O.
5. Gerar pelo menos um ranking do acervo (por execuções, por duração, ou outro critério justificável) usando um algoritmo de ordenação implementado à mão.
6. Apresentar o player num tempo de banca individual, demonstrando as cinco funcionalidades acima com dado real do próprio acervo.

"Funcional antes de bonito" é o padrão de qualidade: a interface continua sendo o menu de texto já existente em main.py — o foco das seis semanas é a estrutura de dados nova de cada semana, não uma reforma visual do programa.

Arquitetura de referência

A mesma separação já praticada desde a Aula 01 continua valendo: cada TAD mora no seu próprio módulo dentro de soundlab/, e main.py apenas orquestra chamadas — nenhuma lógica de estrutura de dados é escrita direto em main.py.

```

soundlab/
├── faixa.py      (Aula 02, sem alteração)
├── acervo.py     (Aula 03, sem alteração)
├── playlist.py  (Aulas 04-05, sem alteração)
├── pilha.py     (Aula 06, sem alteração)
├── fila.py      (Semana 1 — novo)
├── arvore.py    (Semana 2 — novo)
├── arvore_busca.py (Semana 3 — novo)
├── grafo.py     (Semana 4 — novo)
├── busca.py     (Semana 5 — novo)
└── ordenacao.py (Semana 6 — novo)

```

Cada módulo novo soma-se aos anteriores — nenhum é reescrito nas semanas seguintes.

2. Cronograma Geral

Semana	Aula correspondente	Tema	Entrega da semana
1	Aula 07	Fila de Execução e Repetição	Fila ("a seguir") e modo repetição funcionando sobre a playlist
2	Aula 08	Árvore Binária: Índice do Acervo	Árvore binária construída e os três percursos (pré, em, pós-ordem) funcionando
3	Aula 09	Árvore Binária de Busca	Busca por título via ABB e listagem alfabética por percurso em-ordem
4	Aula 10	Grafos e Recomendação	Grafo de coocorrência e recomendação ("quem ouviu X também ouviu Y") funcionando
5	Aula 11	Busca Linear, Binária e Big-O	Comparação numérica entre busca linear e busca binária/ABB, com interpretação
6	Aula 12	Ordenação e Rankings	Pelo menos um ranking do acervo, gerado por ordenação implementada à mão
Final	Encontro de banca	Apresentação Individual	Demonstração ao vivo do player completo (15 min: 10 de demonstração + 5 de perguntas)

Observação: As datas exatas de cada semana seguem o calendário vigente de Estrutura de Dados I (2 encontros por semana) e serão confirmadas pelo professor no início da Semana 1 — a numeração acima é sequencial a partir da próxima aula após a Avaliação N1.

3. Composição da Nota

Conforme o plano de ensino da disciplina, a avaliação combina a prova teórica (N1, já realizada), entregas semanais que acompanham o incremento do sistema, e uma entrega prática final do projeto com apresentação (N2). Este documento operacionaliza exatamente essas duas últimas partes, referentes às seis semanas finais:

Componente	Peso na nota final da disciplina
Entregas semanais (Semanas 1 a 6, 1/6 cada)	20%
Apresentação final do player individual (N2)	40%
(referência) Avaliação N1, já realizada	40%

Os quatro critérios do plano de ensino — organização do código, correção lógica, eficiência da solução e compreensão dos conceitos aplicados — valem para toda entrega semanal; o peso relativo de cada um varia conforme o foco da semana, detalhado na rubrica de cada semana adiante.

Política de entregas — Tudo ou Nada

- Cada semana tem uma data de entrega equivalente ao segundo encontro daquela semana — a nota é computada diretamente pelo histórico de commits da sua branch no GitHub.
- Entrega feita dentro da semana correta recebe a nota cheia daquele componente; a ausência de commit referente à semana resulta em nota zero — não há reposição automática nem acúmulo para a semana seguinte.

- Entregar uma etapa atrasada, porém antes da apresentação final, pode ser aceito a critério do professor com desconto — não é automático, deve ser combinado individualmente antes da próxima semana começar.
- A apresentação final é obrigatória para compor qualquer nota de N2 referente a este projeto — um aluno sem apresentação não valida as entregas semanais anteriores, mesmo que todas tenham sido feitas.
- Cada entrega é estritamente individual, na sua própria branch. Reaproveitar a base comum das Aulas 01-06 é esperado; copiar a solução de um colega é tratado como as demais atividades da disciplina.

Semana 1 — Fila de Execução e Modo Repetição

Corresponde à Aula 07 (Filas e Filas Circulares).

Objetivos de aprendizagem

- Reconhecer que a fila (FIFO) resolve um problema diferente da pilha (LIFO) já construída na Aula 06: "o que toca depois" nunca é "o que acabou de tocar".
- Implementar uma fila encadeada simples e, a partir dela, uma fila circular para o modo de repetição.
- Integrar a nova fila ao main.py existente sem alterar Faixa, Acervo, Playlist ou Pilha.

Aula Expositiva

Roteiro de conteúdo

1. Recapitulação do estado do projeto ao final da Aula 06: Faixa, Acervo, Playlist, Pilha — o que já existe e nunca mais deve ser reescrito do zero.
2. Apresentação do escopo do player individual (as seis funcionalidades da Seção 1) e do cronograma das seis semanas.
3. FIFO vs. LIFO: a Pilha (Aula 06) resolve "desfazer" — o último evento é o primeiro a sair. A fila de execução precisa do oposto: a primeira faixa colocada na fila "a seguir" é a primeira a tocar.
4. Fila circular: quando o modo repetição está ativo, a última posição da fila volta a apontar para a primeira em vez de simplesmente acabar — mesma ideia de nós ligados por ponteiros, agora com o ponteiro final religado ao início.
5. Apresentação do esqueleto de FilaExecucao, com dois TODO explícitos — enfileirar e desenfileirar são o conceito novo da semana; o esqueleto só fornece a classe No e os atributos.

Código Fornecido — esqueleto de fila.py

Fornecido apenas como moldura (classe No, atributos `_frente/``_tras/``_tamanho`) — os métodos enfileirar e desenfileirar são o que cada aluno implementa; são exatamente o conteúdo novo desta semana, por isso não vêm prontos.

```
class No:
    def __init__(self, valor):
        self.valor = valor
        self.proximo = None

class FilaExecucao:
    """Fila de reprodução (FIFO): a próxima faixa a tocar
    é sempre a que está há mais tempo esperando."""

    def __init__(self):
        self._frente = None
        self._tras = None
        self._tamanho = 0

    def enfileirar(self, faixa):
        # TODO: insira um novo nó no FIM da fila (_tras),
        # religando _tras e, se a fila estava vazia, _frente.
        ...

    def desenfileirar(self):
        # TODO: remova e devolva o valor do nó na FRENTE
        # da fila; se a fila ficar vazia, religue _tras.
        ...

    def vazia(self):
        return self._tamanho == 0

    def __len__(self):
        return self._tamanho
```

Aula Guiada

Roteiro guiado

1. Implementar enfileirar e desenfileirar seguindo os TODO do esqueleto, testando com faixas do acervo já existente.
2. Integrar ao menu de main.py: opções para adicionar uma faixa à fila "a seguir" e para tocar a próxima da fila (desenfileirando).
3. Implementar o modo repetição como uma FilaCircular (subclasse ou variante de FilaExecucao): ao desenfileirar a última faixa, ela é automaticamente reenfileirada no fim.
4. Prova de fogo individual: montar uma fila de 3 faixas, ativar o modo repetição, e tocar 5 vezes seguidas, confirmando com o professor que a sequência se repete corretamente.

Perguntas de verificação

- Por que a fila não podia ser só a Pilha da Aula 06, usada ao contrário? (Esperado: o aluno explica que a ordem de saída é estruturalmente diferente — FIFO não é "LIFO lido de trás para frente", é outra estrutura.)
- O que quebraria no modo repetição se `_tras` não fosse religado corretamente ao desenfileirar o último elemento? (Esperado: o aluno percebe que a fila circular pararia de circular, voltando a se comportar como uma fila comum.)

Erros comuns a observar

- Esquecer de atualizar `_frente` quando a fila fica vazia após um desenfileirar — próxima inserção aponta para um nó "fantasma".
- Implementar o modo repetição reaproveitando a Playlist (cursor proxima/anterior) em vez de uma fila nova — perde o sentido pedagógico da semana.

Entrega da Semana 1

- `fila.py` integrado ao projeto, com `FilaExecucao` funcionando.
- Modo repetição (fila circular) demonstrável: a sequência de faixas se repete sem intervenção manual.
- Commit na própria branch com a mensagem "Entrega da Semana 1" (ver Parte 2 — Guia de Entregas).

Critérios de Avaliação — Semana 1

Critério	Peso	Nível pleno esperado
Organização do código	25%	<code>fila.py</code> separado dos demais módulos; nenhuma classe existente foi alterada.
Correção lógica	45%	Enfileirar/desenfileirar respeitam a ordem FIFO; o modo repetição de fato recicla a fila.
Eficiência da solução	15%	Enfileirar e desenfileirar custam $O(1)$, sem percorrer a fila inteira.
Compreensão dos conceitos	15%	O aluno explica a diferença entre FIFO e LIFO com suas próprias palavras.

Semana 2 — Árvore Binária: Índice do Acervo

Corresponde à Aula 08 (Árvores e Árvores Binárias).

Objetivos de aprendizagem

- Entender a árvore binária como estrutura não linear: cada nó tem até dois filhos, abrindo mão da ordem única de listas/pilhas/filas.
- Implementar os três percursos clássicos — pré-ordem, em-ordem e pós-ordem — e reconhecer o que cada um revela sobre a árvore.
- Preparar o terreno conceitual para a Semana 3, em que a árvore ganha uma regra de ordenação e se torna uma árvore binária de busca.

Aula Expositiva

Roteiro de conteúdo

1. Motivação: Acervo.buscar (Aula 03) é uma busca linear, $O(n)$ — examina faixa por faixa até achar ou esgotar a lista. Uma árvore bem organizada pode reduzir drasticamente as comparações necessárias.
2. Árvore binária genérica: um nó, dois filhos (esquerda e direita), sem compromisso de ordem ainda — serve para visualizar a estrutura antes de especializá-la.
3. Os três percursos: pré-ordem (nó, esquerda, direita), em-ordem (esquerda, nó, direita) e pós-ordem (esquerda, direita, nó) — cada um útil para uma pergunta diferente sobre a árvore.
4. Prévia da Semana 3: se a árvore for construída sempre inserindo "menor à esquerda, maior à direita", o percurso em-ordem devolve os elementos em ordem alfabética de título — gancho direto para a árvore binária de busca.

Código Fornecido — esqueleto de `arvore.py`

Fornecido apenas a classe `NodoArvore` e as assinaturas dos três percursos — a lógica de inserção e de cada percurso é o conteúdo novo da semana.

```
class NodoArvore:
    def __init__(self, faixa):
        self.faixa = faixa
        self.esquerda = None
        self.direita = None

class ArvoreBinaria:
    def __init__(self):
        self._raiz = None

    def inserir(self, faixa):
        # TODO: insira 'faixa' em algum lugar da árvore.
        # Nesta semana a regra de posição é livre — o
        # importante é a mecânica de nós e filhos.
        ...

    def pre_ordem(self):
        # TODO: devolva uma lista [nó, esquerda, direita]
        ...

    def em_ordem(self):
        # TODO: devolva uma lista [esquerda, nó, direita]
        ...

    def pos_ordem(self):
        # TODO: devolva uma lista [esquerda, direita, nó]
        ...
```

Aula Guiada

Roteiro guiado

1. Implementar inserir com uma regra simples e própria (ex.: alternar esquerda/direita, ou por ordem de chegada) — ainda sem compromisso de busca eficiente.
2. Implementar os três percursos recursivamente, testando sobre uma árvore pequena (4 a 6 faixas do próprio acervo).
3. Imprimir o resultado dos três percursos lado a lado e discutir com o professor o que cada ordem revela.
4. Integrar ao menu de main.py: uma opção que constrói a árvore a partir do acervo atual e lista o acervo em pré-ordem, em-ordem e pós-ordem.

Perguntas de verificação

- Se eu inserir as mesmas 5 faixas em ordens diferentes, a árvore resultante é sempre igual? (Esperado: não — o formato da árvore depende da ordem de inserção, ao contrário de uma lista.)
- Qual dos três percursos você usaria para apagar a árvore inteira sem perder nenhum nó pelo caminho? (Esperado: pós-ordem — os filhos são tratados antes do próprio nó.)

Erros comuns a observar

- Confundir a ordem dos três percursos (trocar em-ordem por pré-ordem) — pedir para o aluno narrar a recursão em voz alta resolve rápido.
- Implementar os percursos de forma iterativa complicada em vez de recursiva — a recursão é mais direta e é o que a aula expositiva demonstra.

Entrega da Semana 2

- arvore.py integrado, com inserir e os três percursos funcionando sobre o acervo real.
- Opção no menu que exibe os três percursos lado a lado.
- Commit na própria branch com a mensagem "Entrega da Semana 2".

Critérios de Avaliação — Semana 2

Critério	Peso	Nível pleno esperado
Organização do código	25%	arvore.py separado; NodoArvore não exposto fora do módulo.
Correção lógica	45%	Os três percursos devolvem a ordem correta, verificável à mão numa árvore pequena.
Eficiência da solução	15%	Percursos implementados recursivamente, sem percorrer a árvore mais de uma vez por chamada.
Compreensão dos conceitos	15%	O aluno explica a diferença entre os três percursos com um exemplo próprio.

Semana 3 — Árvore Binária de Busca

Corresponde à Aula 09 (ABB e Percursos).

Objetivos de aprendizagem

- Especializar a árvore binária da Semana 2 numa árvore binária de busca (ABB): inserção ordenada por título.
- Implementar busca por título em $O(\log n)$ médio, substituindo (ou complementando) a busca linear de Acervo.
- Usar o percurso em-ordem, já implementado na Semana 2, para obter a listagem alfabética do acervo de graça.

Aula Expositiva

Roteiro de conteúdo

1. A regra que transforma uma árvore binária comum em ABB: ao inserir, todo título menor vai para a esquerda, todo título maior vai para a direita — comparação feita a cada nó, descendo a árvore.
2. Busca por título reaproveita a mesma regra: comparar com o nó atual e decidir ir para a esquerda, para a direita, ou parar (achou) — $O(\log n)$ numa árvore balanceada, $O(n)$ no pior caso (árvore degenerada em lista).
3. Em-ordem numa ABB sempre devolve os elementos em ordem crescente de título — é o percurso da Semana 2, sem nenhuma lógica nova, só aplicado à nova regra de inserção.
4. Gancho para a Semana 5: o pior caso da ABB (árvore degenerada) tem o mesmo custo da busca linear — a eficiência da árvore depende de como os dados chegam.

Aula Guiada

Roteiro guiado

1. Copiar `arvore.py` para `arvore_busca.py` (ou estender a classe existente) e reescrever `inserir` com a regra de comparação por título.
2. Implementar `buscar(titulo)`, descendo a árvore por comparação, devolvendo a Faixa encontrada ou `None`.
3. Popular a ABB com o acervo completo e chamar `em_ordem()` (da Semana 2, sem alteração) para obter a listagem alfabética.
4. Integrar ao menu: uma opção de busca por título via ABB e uma opção de listagem alfabética via em-ordem.
5. Prova de fogo individual: buscar um título que existe, um que não existe, e narrar ao professor o caminho percorrido na árvore em cada caso.

Perguntas de verificação

- Se eu inserir as faixas do acervo já em ordem alfabética, que formato a ABB assume? (Esperado: o aluno percebe que ela degenera numa lista — todo nó só tem filho à direita — e que a busca volta a custar $O(n)$.)
- Por que o em-ordem da Semana 2 não precisou de nenhuma alteração para funcionar aqui? (Esperado: o percurso sempre visita esquerda-nó-direita; o que mudou foi a REGRA DE INSERÇÃO, não o percurso.)

Erros comuns a observar

- Comparar títulos sem normalizar maiúsculas/minúsculas, quebrando a ordenação esperada (mesma armadilha já vista em `Acervo.buscar`).
- Implementar `buscar` reconstruindo uma busca linear disfarçada (percorrendo todos os nós) em vez de descer a árvore por comparação.

Entrega da Semana 3

- `arvore_busca.py` com inserção ordenada e `buscar(titulo)` funcionando em $O(\log n)$ médio.
- Listagem alfabética do acervo via percurso em-ordem.
- Commit na própria branch com a mensagem "Entrega da Semana 3".

Critérios de Avaliação — Semana 3

Critério	Peso	Nível pleno esperado
Organização do código	20%	ABB em módulo próprio, reaproveitando os percursos da Semana 2 sem duplicá-los.
Correção lógica	40%	Busca e inserção respeitam a regra menor-esquerda/maior-direita; listagem alfabética correta.
Eficiência / justificativa de complexidade	25%	O aluno sabe dizer o custo médio ($O(\log n)$) e o pior caso ($O(n)$, árvore degenerada) da busca.
Compreensão dos conceitos	15%	Explica por que em-ordem produz ordem alfabética só na ABB, não na árvore genérica da Semana 2.

Semana 4 — Grafos e Recomendação

Corresponde à Aula 10 (Grafos: Conceitos e Representação).

Objetivos de aprendizagem

- Modelar um grafo simples (faixas como vértices, coocorrência de reprodução como arestas) para um mecanismo de recomendação.
- Implementar a representação por lista de adjacência e uma busca (BFS ou DFS) para encontrar vizinhos a distância 1 ou 2.
- Entender grafo como a estrutura certa quando a relação entre os dados não é linear nem hierárquica, mas uma rede.

Aula Expositiva

Roteiro de conteúdo

1. Por que nem lista, nem pilha/fila, nem árvore resolvem "quem ouviu X também ouviu Y": a relação entre faixas não tem uma ordem única (lista) nem uma hierarquia de pai/filho (árvore) — é uma rede, onde qualquer faixa pode se relacionar com qualquer outra.
2. Modelagem simples: cada vez que duas faixas tocam em sequência direta (via fila de execução, Semana 1, ou via a playlist), cria-se (ou reforça-se) uma aresta entre elas no grafo de coocorrência.
3. Representação por lista de adjacência: cada faixa guarda a lista de faixas com as quais já coocorreu — mais simples de implementar que matriz de adjacência para um acervo pequeno.
4. Recomendação como busca: vizinhos diretos (distância 1) de uma faixa são a recomendação de primeiro nível; aplicar BFS a partir dela por mais um passo alcança recomendações de segundo nível ("quem ouviu quem ouviu X").

Código Fornecido — esqueleto de grafo.py

Fornecido apenas a estrutura de lista de adjacência e a assinatura de recomendar — a lógica de BFS (fila, visitados, laço) é o conteúdo novo, por isso vem com TODO.

```
from soundlab.fila import FilaExecucao

class GrafoRecomendacao:
    """Vértices: títulos de faixas. Arestas: coocorrência
    de reprodução em sequência."""

    def __init__(self):
        self._adjacencia = {} # titulo -> set de titulos vizinhos

    def registrar_coocorrencia(self, titulo_a, titulo_b):
        self._adjacencia.setdefault(titulo_a, set()).add(titulo_b)
        self._adjacencia.setdefault(titulo_b, set()).add(titulo_a)

    def recomendar(self, titulo, distancia_maxima=2):
        """Devolve os titulos alcancaveis a partir de 'titulo'
        em até 'distancia_maxima' saltos, usando BFS."""
        # TODO: use uma FilaExecucao (Semana 1) para visitar
        # os vizinhos em camadas, registrando a distância de
        # cada título visitado a partir da origem. Devolva
        # todos os títulos com distância > 0 e <= distancia_maxima.
        ...
```

Aula Guiada

Roteiro guiado

1. Implementar o TODO de recomendar usando BFS: uma fila (reaproveitando FilaExecucao da Semana 1), um conjunto de visitados, e um dicionário de distância por título.
2. Popular o grafo automaticamente: sempre que proxima()/desenfileirar() tocar uma faixa logo após outra, registrar a coocorrência entre as duas.

3. Testar recomendar com um título do próprio histórico de reprodução e conferir, à mão, se os vizinhos de distância 1 e 2 fazem sentido.
4. Integrar ao menu: uma opção "recomendar a partir de uma faixa", exibindo os títulos sugeridos.

Perguntas de verificação

- Por que usar um set (não uma lista) para os vizinhos de cada título? (Esperado: evita arestas duplicadas quando duas faixas coocorrem mais de uma vez.)
- O que aconteceria se recomendar usasse busca em profundidade (DFS) em vez de BFS? (Esperado: o aluno percebe que DFS não garante visitar primeiro os vizinhos mais próximos — a noção de "distância" exige BFS.)

Erros comuns a observar

- Esquecer de marcar um título como visitado antes de enfileirar seus vizinhos, causando laço infinito em grafos com ciclos.
- Registrar a coocorrência numa única direção ($A \rightarrow B$ mas não $B \rightarrow A$) — o grafo de recomendação deve ser não direcionado.

Entrega da Semana 4

- grafo.py integrado, com registrar_coocorrencia e recomendar (BFS) funcionando.
- Grafo populado a partir de reprodução real (fila/histórico), não dados fictícios fixos no código.
- Commit na própria branch com a mensagem "Entrega da Semana 4".

Critérios de Avaliação — Semana 4

Critério	Peso	Nível pleno esperado
Organização do código	20%	grafo.py separado; reaproveita FilaExecucao da Semana 1 sem duplicar lógica de fila.
Correção lógica	45%	BFS visita vizinhos em camadas corretas; nenhum laço infinito em grafos com ciclos.
Eficiência / justificativa de complexidade	20%	O aluno sabe explicar por que BFS (não DFS) é a escolha certa para "distância".
Compreensão dos conceitos	15%	Explica, com exemplo real do próprio histórico, por que a relação exigia um grafo.

Semana 5 — Busca Linear, Busca Binária e Big-O

Corresponde à Aula 11 (Busca Linear, Busca Binária e Big-O).

Objetivos de aprendizagem

- Comparar formalmente o custo da busca linear original (Acervo.buscar, Aula 03) com uma busca binária sobre dados ordenados.
- Medir, com números reais do próprio acervo, quantas comparações cada abordagem faz — não só estimar teoricamente.
- Relacionar o resultado medido à notação Big-O apresentada no plano de ensino: $O(n)$ vs. $O(\log n)$.

Aula Expositiva

Roteiro de conteúdo

1. Revisão da busca linear já existente (Acervo.buscar, Aula 03): no pior caso, examina o acervo inteiro — $O(n)$.
2. Busca binária sobre uma lista ordenada: a cada comparação, descarta metade do espaço de busca restante — $O(\log n)$, mas exige que os dados estejam ordenados primeiro.
3. Conexão direta com a ABB da Semana 3: a árvore binária de busca é, na prática, uma busca binária "com memória" — mantém a ordenação permanentemente, em vez de reordenar a cada consulta.
4. Por que medir de verdade, não só estimar: apresentação do helper contador_de_comparacoes (fornecido) — um instrumento de medição, não parte da lógica do player.

Código Fornecido — instrumento de medição

Fornecido pronto porque é um instrumento de medição, não conteúdo pedagógico da semana — mesma lógica já aplicada a utilitários de apoio em projetos anteriores da disciplina.

```
def busca_linear_contada(lista, titulo):
    """Busca linear instrumentada: devolve (faixa_ou_None,
    numero_de_comparacoes feitas até achar ou esgotar a lista)."""
    comparacoes = 0
    for faixa in lista:
        comparacoes += 1
        if faixa.titulo.lower() == titulo.lower():
            return faixa, comparacoes
    return None, comparacoes

def busca_binaria_contada(lista_ordenada, titulo):
    """Busca binária instrumentada sobre uma lista JA ORDENADA
    por titulo. Devolve (faixa_ou_None, numero_de_comparacoes)."""
    inicio, fim, comparacoes = 0, len(lista_ordenada) - 1, 0
    while inicio <= fim:
        comparacoes += 1
        meio = (inicio + fim) // 2
        if lista_ordenada[meio].titulo.lower() == titulo.lower():
            return lista_ordenada[meio], comparacoes
        elif lista_ordenada[meio].titulo.lower() < titulo.lower():
            inicio = meio + 1
        else:
            fim = meio - 1
    return None, comparacoes
```

soundlab/busca.py — módulo completo, fornecido ao aluno no início da semana.

Aula Guiada

Roteiro guiado

1. Copiar busca.py para o próprio projeto e rodar as duas funções sobre o acervo atual, buscando títulos variados (início, meio, fim da lista, e um título inexistente).
2. Para a busca binária, obter a lista ordenada a partir do em_ordem() da ABB (Semana 3) — sem reordenar manualmente.

3. Montar uma tabela simples (print ou arquivo de texto) com o número de comparações de cada busca para pelo menos 5 títulos diferentes.
4. Escrever duas ou três frases comparando o resultado medido com a expectativa teórica ($O(n)$ vs. $O(\log n)$) — não precisa de gráfico, só da comparação qualitativa.

Perguntas de verificação

- Para um acervo de 4 faixas, a diferença entre as duas buscas é visível? E para 100? (Esperado: o aluno percebe que a vantagem de $O(\log n)$ cresce com o tamanho do acervo — para poucas faixas, a diferença é irrelevante na prática.)
- Por que a busca binária exige a lista ordenada antes de começar, e a linear não? (Esperado: o aluno entende que o "custo de ordenar" é um preço pago uma vez, enquanto a ABB paga esse preço de forma incremental, à medida que insere.)

Erros comuns a observar

- Rodar `busca_binaria_contada` sobre uma lista que não está de fato ordenada — o resultado é incorreto sem gerar nenhum erro visível, armadilha silenciosa.
- Comparar tempo de execução (`time.perf_counter`) em vez de número de comparações — para acervos pequenos, o tempo é dominado por ruído do sistema, não pelo algoritmo; contar comparações é mais didático aqui.

Entrega da Semana 5

- `busca.py` integrado, com as duas buscas instrumentadas funcionando sobre o acervo real.
- Tabela ou texto com o número de comparações de cada busca para pelo menos 5 títulos.
- Duas ou três frases de interpretação, relacionando o resultado medido a $O(n)$ e $O(\log n)$.
- Commit na própria branch com a mensagem "Entrega da Semana 5".

CrITÉRIOS de Avaliação — Semana 5

CrITÉrio	Peso	NÍvel pleno esperado
Organização do código	15%	<code>busca.py</code> não foi alterado; a comparação é feita num script ou função separada.
Correção lógica	30%	A busca binária só é aplicada sobre dados de fato ordenados; os números de comparação estão corretos.
Eficiência / justificativa de complexidade	35%	O aluno relaciona corretamente os números medidos a $O(n)$ e $O(\log n)$, com pelo menos 5 títulos testados.
Compreensão dos conceitos	20%	Explica por que a vantagem da busca binária depende do tamanho do acervo.

Semana 6 — Ordenação e Rankings

Corresponde à Aula 12 (Ordenação e Análise de Desempenho).

Objetivos de aprendizagem

- Implementar manualmente um algoritmo de ordenação (inserção ou bolha) sobre as Faixas do acervo.
- Gerar pelo menos um ranking do acervo a partir da ordenação implementada — não da função `sorted()` do Python.
- Comparar informalmente o próprio algoritmo implementado com `sorted()`, fechando o ciclo de "implementar manualmente antes de usar biblioteca pronta" seguido desde a Aula 01.

Aula Expositiva

Roteiro de conteúdo

1. Por que ordenar: rankings (mais tocadas, mais longas) são uma pergunta de negócio legítima do player, e exigem reorganizar o acervo por um critério numérico — não alfabético, como a ABB já resolve.
2. Ordenação por inserção: a cada passo, pega o próximo elemento e o insere na posição correta entre os já ordenados — mesma ideia intuitiva de organizar cartas na mão.
3. Custo: $O(n^2)$ no pior caso — cada um dos n elementos pode exigir até n comparações para achar sua posição. Ligação direta com a comparação $O(n)$ vs. $O(\log n)$ da Semana 5.
4. Regra de ouro mantida até o fim: implementar a ordenação à mão primeiro; `sorted()` só é usado depois, como termo de comparação — nunca substitui o algoritmo em avaliação.

Aula Guiada

Roteiro guiado

1. Implementar `ordenar_por_insercao(lista, chave)` em `ordenacao.py`, onde `chave` é uma função que extrai o campo de comparação (ex.: `lambda f: f.num_execucoes`).
2. Gerar pelo menos um ranking real do acervo (por `num_execucoes` ou por `duracao_seg` — ambos já existem em Faixa) usando a ordenação implementada.
3. Comparar o resultado com `sorted(acervo, key=chave)` e confirmar que as duas listas saem idênticas — prova de que a implementação manual está correta.
4. Integrar ao menu: uma opção de ranking, com o critério escolhido pelo aluno.

Perguntas de verificação

- Se o acervo já estivesse quase ordenado, a ordenação por inserção seria mais rápida ou mais lenta que num acervo embaralhado? (Esperado: mais rápida — é um dos poucos algoritmos simples sensíveis à ordem de entrada.)
- Por que comparar com `sorted()` em vez de simplesmente usar `sorted()` no projeto final? (Esperado: `sorted()` é biblioteca pronta — a regra da disciplina exige implementar manualmente a estrutura/algoritmo em estudo antes de usar o equivalente pronto.)

Erros comuns a observar

- Ordenar a lista inteira de Faixas por comparação direta de objetos, sem usar a função `chave` — funciona por acidente só se Faixa implementar comparação, o que não é o caso aqui.
- Confundir `ordenar_por_insercao` com a inserção ordenada já vista na ABB (Semana 3) — são dois algoritmos diferentes, mesmo compartilhando a palavra "inserção".

Entrega da Semana 6

- `ordenacao.py` com `ordenar_por_insercao` (ou outro algoritmo à mão) implementado e testado.
- Pelo menos um ranking do acervo exibido no menu, gerado pela ordenação implementada.
- Confirmação (print ou comentário) de que o resultado coincide com `sorted()` para o mesmo critério.
- Commit na própria branch com a mensagem "Entrega da Semana 6".

Critérios de Avaliação — Semana 6

Critério	Peso	Nível pleno esperado
Organização do código	20%	ordenacao.py separado; critério de ordenação passado como função (chave), não fixo no algoritmo.
Correção lógica	40%	O ranking gerado é idêntico ao de sorted() para o mesmo critério, em qualquer acervo testado.
Eficiência / justificativa de complexidade	25%	O aluno sabe dizer o custo $O(n^2)$ do algoritmo escolhido e por que ele foi suficiente para o tamanho do acervo do projeto.
Compreensão dos conceitos	15%	Explica por que implementar manualmente antes de usar sorted() faz parte do método da disciplina.

Apresentação Final — Banca Individual

Encontro de banca ao final das seis semanas, conforme o calendário confirmado pelo professor.

Roteiro da apresentação

1. Contexto (1 min): o que é o SoundLab e o que o seu player individual acrescentou a ele.
2. Demonstração ao vivo (8-10 min): com faixas reais do seu acervo, mostre a fila/repetição (Semana 1), a busca pela ABB (Semana 3), uma recomendação do grafo (Semana 4) e um ranking (Semana 6).
3. Decisões técnicas (3-4 min): qual critério de recomendação/ranking você escolheu e por quê; o que a comparação de buscas (Semana 5) revelou no seu acervo.
4. Dificuldades e o que ficaria para uma próxima versão (1-2 min).

Critérios de Avaliação — Banca Final (N2)

Critério	Peso	Nível pleno esperado
Funcionalidade do player	35%	As cinco funcionalidades das seis semanas estão presentes e demonstráveis ao vivo, com dado real do próprio acervo.
Correção lógica e robustez	25%	Fila, ABB, grafo e ordenação funcionam corretamente nos casos testados pela banca.
Qualidade técnica e justificativa de complexidade	25%	O aluno explica com segurança o custo de cada estrutura usada e as decisões de projeto tomadas.
Comunicação e autoria	15%	Apresentação clara, dentro do tempo, com domínio real do próprio código ao responder perguntas.

Observação: "Player individual" significa que o critério de recomendação, o critério de ranking e as pequenas decisões de organização são próprios de cada aluno — mas o núcleo (Faixa, Acervo, Playlist, Pilha) é o mesmo SoundLab construído pela turma inteira, e deve ser reconhecível como tal pela banca.

Anexo A — Convenções Já Estabelecidas no Curso

- Nomes de classes, métodos e variáveis em português, consistente com todo o SoundLab desde a Aula 01.
- Nenhuma biblioteca pronta substitui uma estrutura já estudada manualmente (fila, árvore, grafo, ordenação) antes de o aluno tê-la implementado — a regra não se aplica a instrumentos de medição (busca.py, Semana 5), que são infraestrutura, não o conteúdo em avaliação.
- Reaproveitar, não reescrever: Faixa, Acervo, Playlist e Pilha (Aulas 01-06) devem ser chamados pelos novos módulos das seis semanas, nunca duplicados.
- Entregas estritamente individuais a partir da Semana 1 — diferente do ritmo coletivo das Aulas 01 a 06.

Anexo B — Ambiente de Desenvolvimento (Windows)

- Comando para rodar: `python main.py` — Windows não tem o alias `python3` por padrão.
- Nenhum dos módulos novos das seis semanas (fila, árvore, grafo, busca, ordenação) depende de biblioteca externa — tudo é biblioteca padrão do Python, nenhuma instalação adicional é necessária.
- Git for Windows (inclui Git Bash) deve estar instalado antes da Semana 1 — ver Parte 2 deste documento para o passo a passo de clone e branch.

Guia de Entregas — SoundLab Individual

Como clonar, criar sua branch e entregar cada semana via GitHub

Critério de Avaliação: Tudo ou Nada

A nota deste projeto será computada semanalmente via GitHub. O cumprimento do prazo é rigoroso: entregas realizadas dentro da semana correta recebem a nota cheia (100%); a ausência de commits referentes à semana resulta em nota zero (0%). As entregas serão verificadas diretamente pelo histórico da sua branch.

Neste bimestre, utilizaremos o repositório central do projeto SoundLab. Para garantir que todos consigam trabalhar simultaneamente sem que um aluno sobrescreva o código do outro, cada um de vocês trabalhará em sua própria linha do tempo isolada, chamada de Branch.

1. Clonando o Repositório Base

Este passo deve ser feito apenas uma vez, na primeira semana. Abra o terminal (ou Git Bash) no seu computador, navegue até a pasta onde deseja salvar os seus trabalhos da faculdade e execute:

```
git clone https://github.com/Evertronico/SoundLab.git
cd SoundLab
```

2. Criando a sua Branch Exclusiva

Para não interferir no código principal, você criará uma cópia do ambiente com o seu nome e matrícula. Execute este comando apenas uma vez após clonar o projeto (substitua pelo seu nome e matrícula sem espaços):

```
git checkout -b seu-nome-matricula123
```

O comando checkout -b cria a branch e já muda o seu ambiente para ela. A partir deste momento, todas as alterações que você fizer estarão seguras e isoladas no seu perfil.

3. A Rotina de Entrega Semanal

Sempre que você finalizar a atividade programada para a semana, será necessário empacotar suas modificações e enviá-las ao GitHub para avaliação. Siga os três comandos abaixo em ordem:

1. Adicionar as alterações

Avisa o Git sobre os arquivos que você criou ou modificou.

```
git add .
```

2. Registrar o Commit

Cria o "carimbo" oficial da sua entrega. É obrigatório identificar a semana no texto.

```
git commit -m "Entrega da Semana 1"
```

3. Enviar para o GitHub

Transfere o código do seu computador para a nuvem.

Observação: Na sua primeira entrega (Semana 1), o comando de envio é um pouco maior para conectar sua branch local à nuvem:

```
git push -u origin seu-nome-matricula123
```

Da Semana 2 em diante, como o Git já conhecerá o caminho, bastará usar o comando simplificado:

```
git push
```

Como ter certeza de que a entrega deu certo?

1. Acesse o link: <https://github.com/Evertronico/SoundLab>

2. No canto superior esquerdo, clique no botão que diz main.
3. Procure o nome da sua branch na lista.
4. Ao clicar nela, você verá os seus arquivos e a mensagem do commit que acabou de enviar (ex.: "Entrega da Semana 1"). Se o código estiver lá, sua entrega está registrada.

Observação: *Sua branch nunca é enviada automaticamente para main — ela permanece isolada, e é exatamente essa isolamento que garante que o trabalho de um colega nunca sobrescreva o seu.*